

YaRN: Efficient Context Window Extension of Large Language Models

Bowen Peng^{1*} Jeffrey Quesnelle^{1†} Honglu Fan^{2‡} Enrico Shippole[‡]

¹Nous Research ²EleutherAI ³University of Geneva

Abstract

Rotary Position Embeddings (RoPE) have been shown to effectively encode positional information in transformer-based language models. However, these models fail to generalize past the sequence length they were trained on. We present YaRN (Yet another RoPE extension method), a compute-efficient method to extend the context window of such models, requiring 10x less tokens and 2.5x less training steps than previous methods. Using YaRN, we show that LLaMA models can effectively utilize and extrapolate to context lengths much longer than their original pre-training would allow, while also surpassing previous the state-of-the-art at context window extension. In addition, we demonstrate that YaRN exhibits the capability to extrapolate beyond the limited context of a fine-tuning dataset. The models fine-tuned using YaRN has been made available and reproduced online up to 128k context length at <https://github.com/jquesnelle/yarn>.

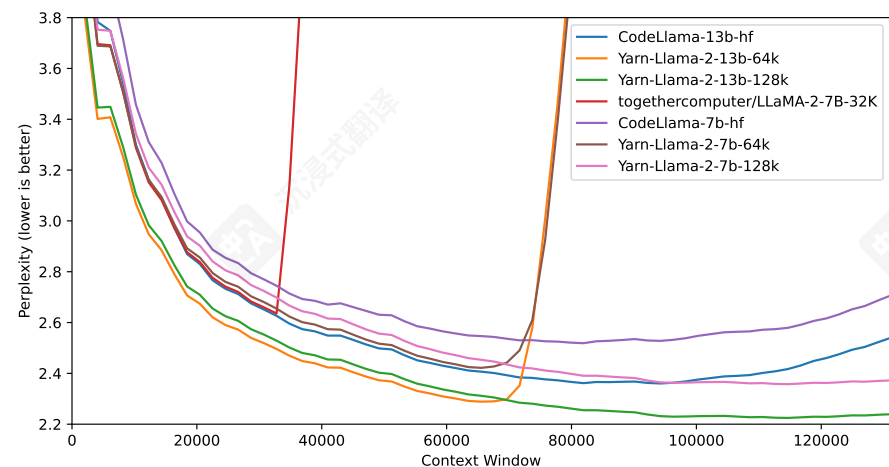


Figure 1: Sliding window perplexity ($S = 256$) of ten 128k Proof-pile documents truncated to evaluation context window size

*Reddit: /u/bloc97 GitHub: bloc97

†Reddit: /u/emozilla X: @theemozilla GitHub: jquesnelle

‡X: @EnricoShippole GitHub: conceptofmind

YaRN：大型语言模型的效率上下文窗口扩展

彭 Bowen^{1*} 钱 Jeffrey^{1†} 范洪路^{2‡} 希普奥利 Enrico[‡]

¹Nous Research ²EleutherAI ³日内瓦大学

摘要

旋转位置嵌入（RoPE）已被证明能有效编码基于 Transformer 的语言模型中的位置信息。然而，这些模型无法泛化到其训练的序列长度之外。我们提出了 YaRN（Yet another RoPE extension method），一种计算高效的方法，用于扩展此类模型的上下文窗口，其所需的标记数量比先前方法少 10 倍，训练步数少 2.5 倍。使用 YaRN，我们展示了 LLaMA 模型可以有效地利用并扩展到比其原始预训练允许的上下文长度长得多的范围，同时在上下文窗口扩展方面也超越了先前最先进技术。此外，我们证明了 YaRN 具有超出微调数据集有限上下文范围的能力。使用 YaRN 微调的模型已在线提供并复现，最高可达 128k 上下文长度，请访问 <https://github.com/jquesnelle/yarn>。

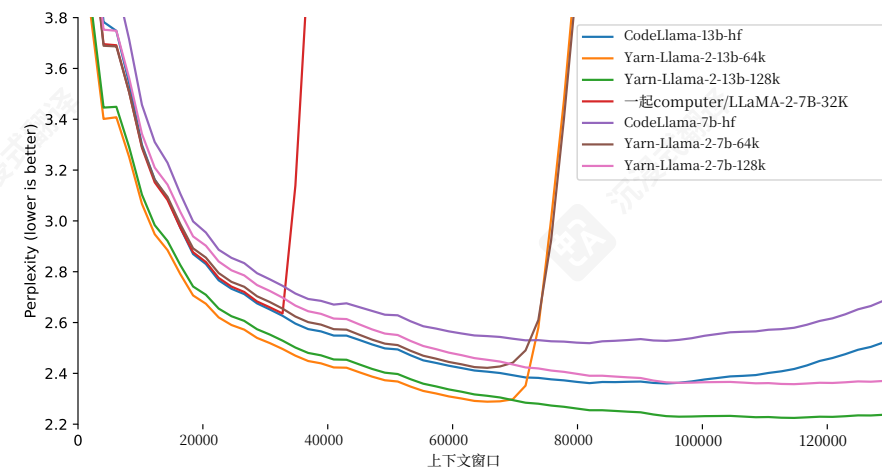


图1：十个 128k 证明堆文档截断到评估上下文窗口大小的滑动窗口困惑度 ($S = 256$)

Reddit: /u/bloc97 GitHub: bloc97

Reddit: /u/emozilla X: @theemozilla GitHub: jquesnelle

‡X: @EnricoShippole GitHub: conceptofmind

1 Introduction

Transformer-based Large Language Models[40] (LLMs) have become the near-ubiquitous choice for many natural language processing (NLP) tasks where long-range abilities such as *in-context learning* (ICL) has been crucial. In performing the NLP tasks, the maximal length of the sequences (the *context window*) determined by its training processes has been one of the major limits of a pretrained LLM. Being able to dynamically extend the context window via a small amount of fine-tuning (or without fine-tuning) has become more and more desirable. To this end, the position encodings of transformers are the center of the discussions.

The original Transformer architecture used an absolute sinusoidal position encoding, which was later improved to a learnable absolute position encoding [15]. Since then, relative positional encoding schemes [32] have further increased the performance of Transformers. Currently, the most popular relative positional encodings are *T5 Relative Bias* [30], *RoPE* [34], *XPos* [35], and *ALiBi* [27].

One reoccurring limitation with positional encodings is the inability to generalize past the context window seen during training. While some methods such as ALiBi are able to do limited generalization, none are able to generalize to sequences significantly longer than their pre-trained length [22].

Some works have been done to overcome such limitation. [9] and concurrently [21] proposed to extend the context length by slightly modifying RoPE via Position Interpolation (PI) and fine-tuning on a small amount of data. As an alternative, [6] proposed the "NTK-aware" interpolation by taking the loss of high frequency into account. Since then, two improvements of the "NTK-aware" interpolation have been proposed, with different emphasis:

- the "Dynamic NTK" interpolation method [14] for pre-trained models without fine-tuning.
- the "NTK-by-parts" interpolation method [7] which performs the best when fine-tuned on a small amount of longer-context data.

The "NTK-aware" interpolation and the "Dynamic NTK" interpolation have already seen their presence in the open-source models such as Code Llama [31] (using "NTK-aware" interpolation) and Qwen 7B [2] (using "Dynamic NTK").

In this paper, in addition to making a complete account of the previous unpublished works on the "NTK-aware", the "Dynamic NTK" and the "NTK-by-part" interpolations, we present YaRN (Yet another RoPE extension method), an improved method to efficiently extend the context window of models trained with Rotary Position Embeddings (RoPE) including the LLaMA [38], the GPT-NeoX [5], and the PaLM [10] families of models.

YaRN reaches state-of-the-art performances in context window extensions after fine-tuning on less than $\sim 0.1\%$ of the original pre-training data. In the meantime, by combining with the inference-time technique called Dynamic Scaling, the Dynamic-YaRN allows for more than 2x context window extension without any fine-tuning.

2 Background and Related Work

2.1 Rotary Position Embeddings

The basis of our work is the Rotary Position Embedding (RoPE) introduced in [34]. We work on a hidden layer where the set of hidden neurons are denoted by D . Given a sequence of vectors $\mathbf{x}_1, \dots, \mathbf{x}_L \in \mathbb{R}^{|D|}$, following the notation of [34], the attention layer first converts the vectors into the query vectors and the key vectors:

$$\mathbf{q}_m = f_q(\mathbf{x}_m, m) \in \mathbb{R}^{|D|}, \mathbf{k}_n = f_k(\mathbf{x}_n, n) \in \mathbb{R}^{|D|}. \quad (1)$$

Next, the attention weights are calculated as

$$\text{softmax}\left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{\sqrt{|D|}}\right), \quad (2)$$

where $\mathbf{q}_m, \mathbf{k}_n$ are considered as column vectors so that $\mathbf{q}_m^T \mathbf{k}_n$ is simply the Euclidean inner product. In RoPE, we first assume that $|D|$ is even and identify the embedding space and the hidden states as

1 引言

基于 Transformer 的大型语言模型[40](LLMs)已成为许多自然语言处理 (NLP) 任务中的近通用选择, 在这些任务中, 长距离能力 (如 上下文学习(ICL)) 至关重要。在执行 NLP 任务时, 由其训练过程确定的序列的最大长度 (即上下文窗口) 一直是预训练大型语言模型的主要限制之一。能够通过少量微调 (或不进行微调) 动态扩展上下文窗口变得越来越受欢迎。为此, Transformer 的位置编码是讨论的中心。

原始的 Transformer 架构使用了绝对正弦位置编码, 后来改进为可学习的绝对位置编码 [15]。自那以后, 相对位置编码方案 [32]进一步提升了 Transformer 的性能。目前, 最流行的相对位置编码是 *T5* 相对偏差 [30], *RoPE* [34], *XPos* [35], 和 *ALiBi* [27]。

位置编码的一个反复出现的局限性是无法泛化超过训练期间看到的上下文窗口。虽然一些方法 (如 ALiBi) 能够进行有限泛化, 但没有方法能够泛化到比其预训练长度显著更长的序列 [22]。

一些工作已经尝试克服这种局限性。 [9] 和同时 [21] 提出通过略微修改 RoPE 并通过位置插值 (PI) 在少量数据上进行微调来扩展上下文长度。作为替代方案, [6] 提出了考虑高频损失的 “NTK-aware” 插值。自那以后, 提出了两种 “NTK-aware” 插值的改进, 各有侧重:

- “动态NTK” 插值方法 [14]用于未经微调的预训练模型。
- “NTK-by-parts” 插值方法 [7] 在少量长上下文数据上进行微调时表现最佳。

“NTK-aware” 插值和 “动态NTK” 插值已经在Code Llama [31] (使用 “NTK-aware” 插值)和Qwen 7B [2] (使用 “Dynamic NTK”)等开源模型中出现过。

在本文中, 除了完整记录之前未发表的关于 “NTK-aware”、“Dynamic NTK” 和 “NTK-by-part” 插值的研究成果外, 我们还提出了YaRN(Yet another RoPE extension method), 这是一种改进的方法, 用于高效扩展使用旋转位置嵌入 (RoPE) 训练的模型的上下文窗口, 包括LLaMA [38]、GPT- NeoX [5], 和PaLM [10] 模型系列。

YaRN 在上下文窗口扩展方面, 经过在少于 $\sim 0.1\%$ 的原始预训练数据上进行微调后, 达到了最先进技术性能。与此同时, 通过与称为动态缩放的推理时技术相结合, Dynamic-YaRN 允许在不进行任何微调的情况下实现超过 2 倍的上下文窗口扩展。

2 背景和相关工作

2.1 旋转位置嵌入

我们的工作基础是 [34]中引入的旋转位置嵌入 (RoPE)。我们在一个隐藏层上工作, 其中隐藏神经元的集合用 D 表示。给定一个向量序列 $\mathbf{x}_1, \dots, \mathbf{x}_L \in \mathbb{R}^{|D|}$, 遵循 [34], 的符号, 注意力层首先将向量转换为查询向量和键向量:

$$\mathbf{q}_m = f_q(\mathbf{x}_m, m) \in \mathbb{R}^{|D|}, \mathbf{k}_n = f_k(\mathbf{x}_n, n) \in \mathbb{R}^{|D|}. \quad (1)$$

接下来, 注意力权重计算为

$$\text{softmax}\left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{\sqrt{|D|}}\right), \quad (2)$$

在 $\mathbf{q}_m, \mathbf{k}_n$ 被视为列向量时, $\mathbf{q}_m^T \mathbf{k}_n$ 简单地是欧几里得内积。在RoPE中, 我们首先假设 $|D|$ 是偶数, 并将嵌入空间和隐藏状态识别为

complex vector spaces:

$$\mathbb{R}^{|D|} \cong \mathbb{C}^{|D|/2}$$

where the inner product $\mathbf{q}^T \mathbf{k}$ becomes the real part of the standard Hermitian inner product $\text{Re}(\mathbf{q}^* \mathbf{k})$. More specifically, the isomorphisms interleave the real part and the complex part

$$((\mathbf{x}_m)_1, \dots, (\mathbf{x}_m)_{|D|}) \mapsto ((\mathbf{x}_m)_1 + i(\mathbf{x}_m)_2, \dots, ((\mathbf{x}_m)_{|D|-1} + i(\mathbf{x}_m)_{|D|})), \quad (3)$$

$$((\mathbf{q}_m)_1, \dots, (\mathbf{q}_m)_{|D|}) \mapsto ((\mathbf{q}_m)_1 + i(\mathbf{q}_m)_2, \dots, ((\mathbf{q}_m)_{|D|-1} + i(\mathbf{q}_m)_{|D|})). \quad (4)$$

To convert embeddings $\mathbf{x}_m, \mathbf{x}_n$ into query and key vectors, we are first given $\mathbb{R}$ -linear operators

$$\mathbf{W}_q, \mathbf{W}_k : \mathbb{R}^{|D|} \rightarrow \mathbb{R}^{|D|}.$$

In complex coordinates, the functions f_q, f_k are given by

$$f_q(\mathbf{x}_m, m) = e^{im\theta} \mathbf{W}_q \mathbf{x}_m, \quad f_k(\mathbf{x}_n, n) = e^{in\theta} \mathbf{W}_k \mathbf{x}_n, \quad (5)$$

where $\theta = \text{diag}(\theta_1, \dots, \theta_{|D|/2})$ is the diagonal matrix with $\theta_d = b^{-2d/|D|}$ and $b = 10000$. This way, RoPE associates each (complex-valued) hidden neuron with a separate frequency θ_d . The benefit of doing so is that the dot product between the query vector and the key vector only depends on the relative distance $m - n$ as follows

$$\langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle_{\mathbb{R}} \quad (6)$$

$$= \text{Re}(\langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle_{\mathbb{C}}) \quad (7)$$

$$= \text{Re}(\mathbf{x}_m^* \mathbf{W}_q^* \mathbf{W}_k \mathbf{x}_n e^{i\theta(m-n)}) \quad (8)$$

$$= g(\mathbf{x}_m, \mathbf{x}_n, m - n). \quad (9)$$

In real coordinates, the RoPE can be written using the following function

$$f_{\mathbf{W}}(\mathbf{x}_m, m, \theta_d) = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_l & -\sin m\theta_l \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_l & \cos m\theta_l \end{pmatrix} \mathbf{W} \mathbf{x}_m,$$

so that

$$f_q = f_{\mathbf{W}_q}, \quad f_k = f_{\mathbf{W}_k}.$$

2.2 Position Interpolation

As language models are usually pre-trained with a fixed context length, it is natural to ask how to extend the context length by fine-tuning on relatively less amount of data. For language models using RoPE as the position embedding, Chen et al. [9], and concurrently kaiokendev [21] proposed the Position Interpolation (PI) to extend the context length beyond the pre-trained limit. While a direct extrapolation does not perform well on sequences $w_1, \dots, w_L$ with L larger than the pre-trained limit, they discovered that interpolating the position indicies within the pre-trained limit works well with the help of a small amount of fine-tuning. Specifically, given a pre-trained language model with RoPE, they modify the RoPE by

$$f'_{\mathbf{W}}(\mathbf{x}_m, m, \theta_d) = f_{\mathbf{W}}\left(\mathbf{x}_m, \frac{mL}{L'}, \theta_d\right), \quad (10)$$

where $L' > L$ is a new context window beyond the pre-trained limit. With the original pre-trained model plus the modified RoPE formula, they fine-tuned the language model further on several orders of magnitude fewer tokens (a few billion in Chen et al. [9]) and successfully achieved context window extension.

复向量空间:

$$\mathbb{R}^{|D|} \cong \mathbb{C}^{|D|/2}$$

在内积 $\mathbf{q}^T \mathbf{k}$ 成为标准厄米内积 $\text{Re}(\mathbf{q}^* \mathbf{k})$ 的实部的地方。更具体地说，同构交替了实部和复部

$$((\mathbf{x}_m)_1, \dots, (\mathbf{x}_m)_{|D|}) \mapsto ((\mathbf{x}_m)_1 + i(\mathbf{x}_m)_2, \dots, ((\mathbf{x}_m)_{|D|-1} + i(\mathbf{x}_m)_{|D|})), \quad (3)$$

$$((\mathbf{q}_m)_1, \dots, (\mathbf{q}_m)_{|D|}) \mapsto ((\mathbf{q}_m)_1 + i(\mathbf{q}_m)_2, \dots, ((\mathbf{q}_m)_{|D|-1} + i(\mathbf{q}_m)_{|D|})). \quad (4)$$

要将嵌入 $\mathbf{x}_m, \mathbf{x}_n$ 转换为查询向量和键向量，我们首先被给出 $\mathbb{R}$ -线性算子

$$\mathbf{W}_q, \mathbf{W}_k : \mathbb{R}^{|D|} \rightarrow \mathbb{R}^{|D|}.$$

在复坐标中，函数 f_q, f_k 由给出

$$f_q(\mathbf{x}_m, m) = e^{im\theta} \mathbf{W}_q \mathbf{x}_m, \quad f_k(\mathbf{x}_n, n) = e^{in\theta} \mathbf{W}_k \mathbf{x}_n, \quad (5)$$

其中 $\theta = \text{diag}(\theta_1, \dots, \theta_{|D|/2})$ 是对角矩阵，具有 $\theta_d = b^{-2d/|D|}$ 和 $b = 10000$ 。这样，RoPE将每个(复值的)隐藏神经元与一个单独的频率 θ_d 相关联。这样做的好处是，查询向量和键向量之间的点积仅取决于相对距离 $m - n$ ，如下所示

$$\langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle_{\mathbb{R}} \quad (6)$$

$$= \text{Re}(\langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle_{\mathbb{C}}) \quad (7)$$

$$= \text{Re}(\mathbf{x}_m^* \mathbf{W}_q^* \mathbf{W}_k \mathbf{x}_n e^{i\theta(m-n)}) \quad (8)$$

$$= g(\mathbf{x}_m, \mathbf{x}_n, m - n) \quad (9)$$

在真实坐标中，RoPE 可以使用以下函数表示

$$f_{\mathbf{W}}(\mathbf{x}_m, m, \theta_d) = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_l & -\sin m\theta_l \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_l & \cos m\theta_l \end{pmatrix} \mathbf{W} \mathbf{x}_m,$$

以便

$$f_q = f_{\mathbf{W}_q}, \quad f_k = f_{\mathbf{W}_k}.$$

2.2 位置插值

由于语言模型通常使用固定的上下文长度进行预训练，因此很自然地会问如何通过在相对较少的数据上进行微调来扩展上下文长度。对于使用 RoPE 作为位置嵌入的语言模型，陈等人 [9]，与 kaiokendev [21] 同时提出了位置插值 (PI) 来扩展上下文长度，使其超过预训练限制。虽然直接外推在序列 $w_1, \dots, w_L$ 中表现不佳，其中 L 大于预训练限制，但他们发现，在预训练限制内插值位置索引，并辅以少量微调，效果良好。具体来说，给定一个使用 RoPE 的预训练语言模型，他们通过以下方式修改 RoPE:

$$f'_{\mathbf{W}}(\mathbf{x}_m, m, \theta_d) = f_{\mathbf{W}}\left(\mathbf{x}_m, \frac{mL}{L'}, \theta_d\right), \quad (10)$$

where $L' > L$ 是一个超出预训练限制的新上下文窗口。他们使用原始预训练模型加上修改后的RoPE公式，在数量级更少的标记（陈等人 [9]中的几十亿）上进一步微调语言模型，并成功实现了上下文窗口扩展。

2.3 Additional Notation

The ratio between the extended context length and the original context length has been of special importance, and we introduce the notation s defined by

$$s = \frac{L'}{L}, \quad (11)$$

and we call s the *scale factor*.

We also rewrite and simplify Eq. 10 into the following general form:

$$f'_{\mathbf{W}}(\mathbf{x}_m, m, \theta_d) = f_{\mathbf{W}}(\mathbf{x}_m, g(m), h(\theta_d)), \quad (12)$$

where $g(m), h(\theta_d)$ are method-dependent functions. For PI, we have $g(m) = m/s, h(\theta_d) = \theta_d$. In the subsequent sections, when we introduce a new interpolation method, we sometimes only specify the functions $g(m)$ and $h(\theta_d)$.

Additionally, we define λ_d as the *wavelength* of the RoPE embedding at d -th hidden dimension:

$$\lambda_d = \frac{2\pi}{\theta_d} = 2\pi b^{\frac{2d}{|D|}}. \quad (13)$$

The wavelength describes the length of tokens needed in order for the RoPE embedding at dimension d to perform a full rotation (2π).

Given that some interpolation methods (eg. PI) do not care about the wavelength of the dimensions, we will refer to those methods as "blind" interpolation methods, while others do (eg. YaRN), which we will classify as "targeted" interpolation methods.

2.4 Related work

ReRoPE [33] also aims to extend the context size of existing models pre-trained with RoPE, and claims "infinite" context length without needing any fine-tuning. This claim is backed by a monotonically decreasing loss with increasing context length up to 16k on the Llama 2 13B model. It achieves context extension by modifying the attention mechanism and thus is not purely an embedding interpolation method. Since it is currently not compatible with Flash Attention 2 [13] and requires two attention passes during inference, we do not consider it for comparison.

Concurrently with our work, LM-Infinite [16] proposes similar ideas to YaRN, but focuses on "on-the-fly" length generalization for non-fine-tuned models. Since they also modify the attention mechanism of the models, it is not an embedding interpolation method and is not immediately compatible with Flash Attention 2.

3 Methodology

Whereas PI stretches all RoPE dimensions equally, we find that the theoretical interpolation bound described by PI [9] is insufficient at predicting the complex dynamics between RoPE and the LLM's internal embeddings. In the following subsections, we describe the main issues with PI we have individually identified and solved, so as to give the readers the context, origin and justifications of each method which we use in concert to obtain the full YaRN method.

3.1 Loss of High Frequency information - "NTK-aware" interpolation

If we look at RoPE only from an information encoding perspective, it was shown in [36], using Neural Tangent Kernel (NTK) theory, that deep neural networks have trouble learning high frequency information if the input dimension is low and the corresponding embeddings lack high frequency components. Here we can see the similarities: a token's positional information is one-dimensional, and RoPE expands it to an n-dimensional complex vector embedding.

2.3 附加符号

扩展上下文长度与原始上下文长度的比率一直非常重要，我们引入由 s 定义的符号

$$s = \frac{L'}{L}, \quad (11)$$

我们称 s 为`缩放因子`。

我们将公式10重写并简化为以下通用形式：

$$f'_{\mathbf{W}}(\mathbf{x}_m, m, \theta_d) = f_{\mathbf{W}}(\mathbf{x}_m, g(m), h(\theta_d)), \quad (12)$$

其中 $g(m)$ $h(\theta_d)$ 是方法相关函数。对于PI，我们有 $g(m) = m/s$ $h(\theta_d) = \theta_d$ ，
在后续章节中，当我们引入一种新的插值方法时，我们有时仅指定函数 $g(m)$ 和 $h(\theta_d)$ 。

此外，我们将 λ_d 定义为RoPE嵌入在 d -th隐藏维度处的`波长`：

$$\lambda_d = \frac{2\pi}{\theta_d} = 2\pi b^{\frac{2d}{|D|}}. \quad (13)$$

波长描述了在维度 d 处的RoPE嵌入执行完整旋转(2π)所需的标记长度。

鉴于某些插值方法（例如PI）不关心维度的波长，我们将这些方法称为“盲目”插值方法，而其他方法（例如YaRN）则会关心，我们将这些方法归类为“目标”插值方法。

2.4 相关工作

ReRoPE [33] 也旨在扩展使用RoPE预训练的现有模型的上下文大小，并声称无需任何微调即可实现“无限”上下文长度。这一说法在有16k上下文长度时在Llama 2 13B模型上得到了单调递减的损失支持。它通过修改注意力机制来实现上下文扩展，因此不仅仅是一种嵌入插值方法。由于它目前与Flash Attention 2 [13]不兼容，并且在推理过程中需要两次注意力传递，因此我们不将其纳入比较。

在我们工作的同时，LM-Infinite [16] 提出了与YaRN类似的思路，但专注于非微调模型的“即时”长度泛化。由于它们也修改了模型的注意力机制，因此不是嵌入插值方法，并且与Flash Attention 2不立即兼容。

3 方法论

而PI会平等地拉伸所有RoPE维度，我们发现由PI [9] 描述的理论插值界限在预测RoPE和LLM内部嵌入之间复杂的动态关系时是不够的。在以下小节中，我们描述了我们单独识别并解决的PI的主要问题，以便为读者提供每个我们协同使用以获得完整YaRN方法的方法的背景、来源和理由。

3.1 高频信息丢失 - "NTK-aware"插值

如果我们仅从信息编码的角度来看RoPE，在 [36], 使用神经切线核（NTK）理论中，已经表明如果输入维度较低并且相应的嵌入缺乏高频分量，深度神经网络在学习高频信息时会遇到困难。这里我们可以看到相似之处：token位置信息是一维的，而RoPE将其扩展为n维复向量嵌入。

RoPE closely resembles Fourier Features [36] in many aspects, as it is possible to define RoPE as a special 1D case of a Fourier Feature. Stretching the RoPE embeddings indiscriminately results in the loss of important high frequency details which the network needs in order to resolve tokens that are both very similar and very close together (the rotation describing the smallest distance needs to not be too small for the network to be able to detect it).

We hypothesise that the slight increase of perplexity for short context sizes after fine-tuning on larger context sizes seen in PI [9] might be related to this problem. Under ideal circumstances, there is no reason that fine-tuning on larger context sizes should degrade the performance of smaller context sizes.

In order to resolve the problem of losing high frequency information when interpolating the RoPE embeddings, the "NTK-aware" interpolation was developed in [6]. Instead of scaling every dimension of RoPE equally by a factor s , we spread out the interpolation pressure across multiple dimensions by scaling high frequencies less and low frequencies more. One can obtain such a transformation in many ways, but the simplest would be to perform a base change on the value of θ .

More precisely, following the notations set out in Section 2.3, we define the "NTK-aware" interpolation scheme as follows (see the Appendix A.1 for the details of the deduction).

Definition 1 *The "NTK-aware" interpolation is a modification of RoPE by using Eq. 12 with the following functions.*

$$g(m) = m \quad (14)$$

$$h(\theta_d) = b'^{-2d/|D|}, \quad (15)$$

where

$$b' = b \cdot s^{\frac{|D|}{|D|-2}}. \quad (16)$$

Given the results from [6], this method performs much better at extending the context size of non-fine-tuned models compared to PI [9]. However, one major disadvantage of this method is that given it is not just an interpolation scheme, some dimensions are slightly extrapolated to "out-of-bound" values, thus fine-tuning with "NTK-aware" interpolation [6] yields inferior results to PI [9]. Furthermore, due to the "out-of-bound" values, the theoretical scale factor s does not accurately describe the true context extension scale. In practice, the scale value s has to be set higher than the expected scale for a given context length extension.

We note that shortly before the release of this article, Code Llama [31] was released and uses "NTK-aware" scaling by manually scaling the base b to 1M.

3.2 Loss of Relative Local Distances - "NTK-by-parts" interpolation

In the case of blind interpolation methods like PI and "NTK-aware" interpolation, we treat all the RoPE hidden dimensions equally (as in they have the same effect on the network). However, there are strong clues that point us towards the need for targeted interpolation methods.

In this section, we think heavily in terms of the wavelengths λ_d defined in Eq. 13 in the formula of RoPE. For simplicity, we omit the subscript d in λ_d and the reader is encouraged to think about λ as the wavelength of an arbitrary periodic function.

One interesting observation of RoPE embeddings is that given a context size L , there are some dimensions d where the wavelength is longer than the maximum context length seen during pretraining ($\lambda > L$), this suggests that some dimensions' embeddings might not be distributed evenly in the rotational domain. In such cases, we presume having all unique position pairs implies that the absolute positional information remains intact. On the contrary, when the wavelength is short, only relative positional information is accessible to the network.

Moreover, when we stretch all the RoPE dimensions either by a scale s or using a base change b' , all tokens become closer to each other, as the dot product of two vectors rotated by a lesser amount is bigger. This scaling severely impairs a LLM's ability to understand small and local relationships between its internal embeddings. We hypothesize that such compression leads to the model being confused on the positional order of close-by tokens, and consequently harming the model's abilities.

RoPE在许多方面与傅里叶特征 [36] 非常相似，因为RoPE可以定义为一个傅里叶特征的1D特例。无差别地拉伸RoPE嵌入会导致网络丢失重要的高频细节，而这些细节对于网络分辨非常相似且非常接近的token是必要的（描述最小距离的旋转不能太小，否则网络无法检测到它）。

我们假设，在PI [9] 中，经过在较大上下文大小上进行微调后，短上下文大小下困惑度的轻微增加可能与这个问题有关。在理想情况下，没有理由认为在较大上下文大小上进行微调会降低较小上下文大小的性能。

为了解决在插值RoPE嵌入时丢失高频信息的问题，在 [6]中开发了"NTK-aware"插值。与通过因子 s 等比例缩放RoPE的每个维度不同，我们将插值压力分布在多个维度上，通过减少高频的缩放和增加低频的缩放。可以通过多种方式获得这种变换，但最简单的方法是在 θ 的值上执行基变换。

更精确地说，根据第2.3节中设定的符号，我们将"NTK-aware"插值方案定义如下（见附录A.1，详细了解推导过程）。

定义1 "NTK-aware"插值是通过RoPE使用公式12和以下函数进行修改。

$$g(m) = m \quad (14)$$

$$h(\theta_d) = b'^{-2d/|D|}, \quad (15)$$

其中

$$b' = b \cdot s^{\frac{|D|}{|D|-2}}. \quad (16)$$

根据 [6], 此方法的结果，在扩展非微调模型的上下文大小方面表现远优于 PI [9]。然而，此方法的主要缺点在于，由于它不仅仅是一种插值方案，某些维度会略微外推到“超出范围”的值，因此使用“NTK-aware”插值 [6] 得到的结果劣于 PI [9]。此外，由于“超出范围”的值，理论缩放因子 s 无法准确描述真正的上下文扩展尺度。在实践中，对于给定的上下文长度扩展，缩放值 s 必须设置得高于预期的缩放值。

我们注意到，在本文发布前不久，Code Llama [31] 发布了，它使用“NTK-aware”缩放，通过手动将基础 b 缩放到 1M。

3.2 相对局部距离的丢失 - “NTK-by-parts” 插值

对于PI和"NTK-aware"插值等盲插值方法，我们将所有RoPE隐藏维度同等对待（即它们对网络的影响相同）。然而，有强有力的线索表明我们需要目标插值方法。

在本节中，我们主要考虑RoPE公式中Eq. 13定义的波长 λ_d 。为简化起见，我们省略 λ_d 中的下标 d ，并鼓励读者将 λ 视为任意周期函数的波长。

RoPE嵌入的一个有趣观察是，给定上下文大小 L ，某些维度 d 的波长比预训练($\lambda > L$)期间看到的最大上下文长度更长，这表明某些维度的嵌入可能不会在旋转域中均匀分布。在这种情况下，我们假定所有唯一的位置对意味着绝对位置信息保持完整。相反，当波长较短时，网络只能访问相对位置信息。

此外，当我们通过尺度 s 或使用基变换 b' 拉伸所有 RoPE 维度时，所有标记都会彼此靠近，因为两个向量旋转较小的点积更大。这种缩放严重损害了 LLM 理解其内部嵌入之间小范围局部关系的能力。我们假设这种压缩会导致模型在邻近标记的位置顺序上感到困惑，并因此损害模型的能力。

In order to remedy this issue, given the two previous observations that we have found, we choose not to interpolate the higher frequency dimensions at all while always interpolating the lower frequency dimensions. In particular,

- if the wavelength λ is much smaller than the context size L , we do not interpolate;
- if the wavelength λ is equal to or bigger than the context size L , we want to only interpolate and avoid any extrapolation (unlike the previous "NTK-aware" method);
- dimensions in-between can have a bit of both, similar to the "NTK-aware" interpolation.

As a result, it is more convenient to introduce the ratio $r = \frac{L}{\lambda}$ between the original context size L and the wavelength λ . In the d -th hidden state, the ratio r depends on d in the following way:

$$r(d) = \frac{L}{\lambda_d} = \frac{L}{2\pi b' \frac{2d}{|D|}}. \quad (17)$$

In order to define the boundary of the different interpolation strategies as above, we introduce two extra parameters α, β . All hidden dimensions d where $r(d) < \alpha$ are those where we linearly interpolate by a scale s (exactly like PI, avoiding any extrapolation), and the d where $r(d) > \beta$ are those where we do not interpolate at all. Define the ramp function γ to be

$$\gamma(r) = \begin{cases} 0, & \text{if } r < \alpha \\ 1, & \text{if } r > \beta \\ \frac{r - \alpha}{\beta - \alpha}, & \text{otherwise.} \end{cases} \quad (18)$$

With the help of the ramp function, the "NTK-by-parts" method can be described as follows.

Definition 2 The "NTK-by-parts" interpolation is a modification of RoPE by using Eq. 12 with the following functions⁴.

$$g(m) = m \quad (19)$$

$$h(\theta_d) = \left(1 - \gamma(r(d))\right) \frac{\theta_d}{s} + \gamma(r(d)) \theta_d. \quad (20)$$

The values of α and β should be tuned on a case-by-case basis. For example, we have found experimentally that for the Llama family of models, good values for α and β are $\alpha = 1$ and $\beta = 32$.

Using the techniques described in this section, a variant of the resulting method was released under the name "NTK-by-parts" interpolation [7]. This improved method performs better than the previous PI [9] and "NTK-aware" 3.1 interpolation methods, both with non-fine-tuned models and with fine-tuned models, as shown in [7].

3.3 Dynamic Scaling - "Dynamic NTK" interpolation

In a lot of use cases, multiple forward-passes are performed with varying sequence lengths from 1 to the maximal context size. A typical example is the autoregressive generation where the sequence lengths increment by 1 after each step. There are two ways of applying an interpolation method that uses a scale factor s (including PI, "NTK-aware" and "NTK-by-parts"):

1. Throughout the whole inference cycle, the embedding layer is fixed including the scale factor $s = L'/L$ where L' is the fixed number of extended context size.
2. In each forward-pass, the position embedding updates the scale factor $s = \max(1, l'/L)$ where l' is the sequence length of the current sequence.

The problem of (1) is that the model may experience a performance discount at a length less than L and an abrupt degradation when the sequence length is longer than L' . But by doing Dynamic

⁴The interpolation by linear ramp on h may have alternatives, such as a harmonic mean over θ_d/s and θ_d converted from a linear interpolation on wavelengths. The choice of h here was for the simplicity of implementation, but both would work.

为了解决这个问题，鉴于我们发现的两个先前观察结果，我们选择完全不对高频维度进行插值，而始终对低频维度进行插值。具体来说，

- 如果波长 λ 远小于上下文大小 L ，我们不插值；
- 如果波长 λ 等于或大于上下文大小 L ，我们希望仅插值并避免任何外推（与之前的 "NTK-aware" 方法不同）；

- 中间的维度可以同时具有一些，类似于 "NTK-aware" 插值。

因此，引入原始上下文大小 L 和波长 λ 之间的比率 $r = \frac{L}{\lambda}$ 更方便。在 d -th 隐藏状态中，比率 r 取决于 d ，如下所示：

$$r(d) = \frac{L}{\lambda_d} = \frac{L}{2\pi b' \frac{2d}{|D|}}. \quad (17)$$

为了定义上述不同插值策略的边界，我们引入两个额外参数 α, β 。所有隐藏维度 d ，其中 $r(d) < \alpha$ 是那些我们通过尺度 s 线性插值（与 PI 完全相同，避免任何外推）的维度，而 d ，其中 $r(d) > \beta$ 是那些我们完全不插值的维度。定义斜坡函数 γ 为

$$\gamma(r) = \begin{cases} 0, & \text{if } r < \alpha \\ 1, & \text{if } r > \beta \\ \frac{r - \alpha}{\beta - \alpha}, & \text{otherwise.} \end{cases} \quad (18)$$

借助斜坡函数，"NTK-by-parts"方法可以描述如下。

定义2 "NTK-by-parts"插值是通过RoPE使用公式12并结合以下函数进行的修改⁴。

$$g(m) = m \quad (19)$$

$$h(\theta_d) = \left(1 - \gamma(r(d))\right) \frac{\theta_d}{s} + \gamma(r(d)) \theta_d. \quad (20)$$

α 和 β 的值应根据具体情况调整。例如，我们通过实验发现，对于Llama系列模型， α 和 β 的良好值分别是 $\alpha = 1$ 和 $\beta = 32$ 。

使用本节所述的技术，一种改进后的方法以"NTK-by-parts"插值 [7]的名称发布。这种改进方法在非微调模型和微调模型上均优于之前的PI [9] 和"NTK-aware" 3.1插值方法，如图 [7]所示。

3.3 动态缩放 - "动态NTK"插值

在许多用例中，会使用从1到最大上下文大小的不同序列长度进行多次前向传递。一个典型的例子是自回归生成，其中序列长度在每一步后增加1。应用使用缩放因子 s 的插值方法（包括PI、"NTK-aware"和"NTK-by-parts"）有两种方式：

1. 在整个推理周期中，嵌入层是固定的，包括缩放因子 $s = L'/L$ ，其中 L' 是扩展上下文大小的固定数量。
2. 在每次前向传递中，位置嵌入更新缩放因子 $s = \max(1, l'/L)$ ，其中 l' 是当前序列的序列长度。

问题 (1) 在于模型在序列长度小于 L 时可能会出现性能折扣，而当序列长度大于 L' 时会急剧退化。但通过动态

⁴在 h 上的线性斜坡插值可能有其他替代方案，例如，通过线性插值在波长上从 θ_d/s 和 θ_d 转换得到的调和平均。这里选择 h 是为了实现的简单性，但两者都会起作用。

Scaling as (2), it allows the model to gracefully degrade instead of immediately breaking when hitting the trained context limit L' . We call this inference-time method the Dynamic Scaling method. When it is combined with "NTK-aware" interpolation, we call it "Dynamic NTK" interpolation. It first appeared in public as a reddit post in [14].

One notable fact is that the "Dynamic NTK" interpolation works exceptionally well on models pre-trained on L without any finetuning ($L' = L$). This is supported by the experiment in Appendix B.3.

Often in the repeated forward-passes, the kv-caching [8] is applied so that we can reuse the previous key-value vectors and improve the overall efficiency. We point out that in some implementations when the RoPE embeddings are cached, some care has to be taken in order to modify it for Dynamic Scaling with kv-caching. The correct implementation should cache the kv-embeddings before applying RoPE, as the RoPE embedding of every token changes when s changes.

3.4 YaRN

In addition to the previous interpolation techniques, we also observe that introducing a temperature t on the logits before the attention softmax has a uniform impact on perplexity regardless of the data sample and the token position over the extended context window (See Appendix A.2). More precisely, instead of Eq. 2, we modify the computation of attention weights into

$$\text{softmax} \left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{t\sqrt{|D|}} \right). \quad (21)$$

The reparametrization of RoPE as a set of 2D matrices has a clear benefit on the implementation of this attention scaling: we can instead use a "length scaling" trick which scales both $\mathbf{q}_m$ and $\mathbf{k}_n$ by a constant factor $\sqrt{1/t}$ by simply scaling the complex RoPE embeddings by the same amount. With this, YaRN can effectively alter the attention mechanism without modifying its code. Furthermore, it has zero overhead during both inference and training, as RoPE embeddings are generated in advance and are reused for all forward passes. Combining it with the "NTK-by-parts" interpolation, we have the YaRN method.

Definition 3 By the "YaRN method", we refer to a combination of the attention scaling in Eq. 21 and the "NTK-by-parts" interpolation introduced in Section 3.2.

For LLaMA and Llama 2 models, we recommend the following values:

$$\sqrt{\frac{1}{t}} = 0.1 \ln(s) + 1. \quad (22)$$

The equation above is found by fitting $\sqrt{1/t}$ at the lowest perplexity against the scale extension by various factors s using the "NTK-by-parts" method (Section 3.2) on LLaMA 7b, 13b, 33b and 65b models without fine-tuning. We note that the same values of t also apply fairly well to Llama 2 models (7b, 13b and 70b). It suggests that the property of increased entropy and the temperature constant t may have certain degree of "universality" and may be generalizable across some models and training data.

The YaRN method combines all our findings and surpasses all previous methods in both fine-tuned and non-fine-tuned scenarios. Thanks to its low footprint, YaRN allows for direct compatibility with libraries that modify the attention mechanism such as Flash Attention 2 [13].

4 Experiments

We show that YaRN successfully achieves context window extension of language models using RoPE as its position embedding. Moreover, this result is achieved with only 400 training steps, representing approximately 0.1% of the model's original pre-training corpus, a 10x reduction from Rozière et al. [31] and 2.5x reduction in training steps from Chen et al. [9], making it highly compute-efficient for training with no additional inference costs. We calculate the perplexity of long documents and score

缩放作为 (2), 它允许模型在达到训练的上下文限制 L' 时优雅地退化, 而不是立即崩溃。我们称这种方法为动态缩放方法。当它与“NTK-aware”插值结合时, 我们称其为“动态 NTK”插值。它首次以公开形式出现在 [14] 的一个reddit帖子中。

一个值得注意的事实是, "动态NTK"插值在预训练于 L 的模型上 (未经任何微调 ($L' = L$)) 表现异常出色。附录B.3中的实验支持这一结论。

在重复的前向传递中, 通常应用kv缓存 [8], 以便重用先前的键值向量并提高整体效率。我们指出, 在某些实现中, 当RoPE嵌入被缓存时, 需要特别注意如何对其进行动态缩放与kv缓存的结合。正确的实现应该在应用RoPE之前缓存kv嵌入, 因为当 s 变化时, 每个token的RoPE嵌入都会改变。

3.4 YaRN

除了之前提到的插值技术外, 我们还观察到, 在注意力softmax之前对logits引入温度 t , 对困惑度的影响是均匀的, 无论数据样本和扩展上下文窗口中的token位置如何 (参见附录 A.2)。更精确地说, 我们修改了注意力权重的计算, 而不是Eq. 2, 而是将其改为

$$\text{softmax} \left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{t\sqrt{|D|}} \right). \quad (21)$$

将RoPE重新参数化为一组2D矩阵, 对注意力缩放的实现具有明显的优势: 我们可以使用一个“长度缩放”技巧, 通过简单地以相同的量缩放复数RoPE嵌入, 来对 $\mathbf{q}_m$ 和 $\mathbf{k}_n$ 进行常数因子 $\sqrt{1/t}$ 的缩放。通过这种方式, YaRN可以有效地改变注意力机制, 而无需修改其代码。此外, 它在推理和训练期间都没有额外的开销, 因为RoPE嵌入是预先生成的, 并且用于所有前向传递。结合“NTK-by-parts”插值, 我们得到了YaRN方法。

定义3 根据“YaRN方法”, 我们指的是等式21中的注意力缩放和第3.2节中引入的“NTK-by-parts”插值的组合。

对于LLaMA和Llama 2模型, 我们推荐以下值:

$$\sqrt{\frac{1}{t}} = 0.1 \ln(s) + 1. \quad (22)$$

上述方程是通过使用“NTK-by-parts”方法 (第3.2节) 在未经微调的LLaMA 7b、13b、33b和65b模型上, 将 $\sqrt{1/t}$ 拟合到最低困惑度, 并使用各种因子 s 进行尺度扩展得到的。我们注意到, 相同的 t 值也相当好地适用于Llama 2模型 (7b、13b和70b)。这表明增加熵的性质和温度常数 t 可能具有一定的“普遍性”, 并且可能跨某些模型和训练数据具有可推广性。

YaRN方法结合了我们的所有发现, 并在微调和非微调场景中均超越了所有先前方法。得益于其低占用, YaRN允许与修改注意力机制的库 (如Flash Attention 2 [13]) 直接兼容。

4 实验

我们展示了YaRN成功使用RoPE作为其位置嵌入实现了语言模型的上下文窗口扩展。此外, 这一结果仅通过400个训练步数就实现了, 这大约占模型原始预训练语料库的0.1%, 与Rozière等人[31]相比减少了10倍, 与Chen等人 [9], 相比训练步数减少了2.5倍, 使其在训练时具有极高的计算效率, 且没有额外的推理成本。我们计算了长文档的困惑度并评分

on established benchmarks to evaluate the resulting models, finding that they surpass all other context window extension methods.

We broadly followed the training and evaluation procedures as outlined in [9].

4.1 Training

For training, we extended the Llama 2 [39] 7B and 13B parameter models. No changes were made to the LLaMA model architecture other than the calculation of the embedding frequencies as described in 3.4 with $s = 16$ and $s = 32$.

We used a learning rate of 2×10^{-5} with no weight decay and a linear warmup of 20 steps along with AdamW [24] $\beta_1 = 0.9$ and $\beta_2 = 0.95$. For $s = 16$ we fine-tuned for 400 steps with global batch size 64 using PyTorch [26] Fully Sharded Data Parallelism [42] and Flash Attention 2 [13] on the PG19 dataset [29] chunked into 64k segments bookended with the BOS and EOS token. For $s = 32$ we followed the same procedure, but started from the finished $s = 16$ checkpoint and trained for an additional 200 steps.

4.2 Extrapolation and Transfer Learning

In Code Llama [31], a dataset with 16k context was used with a scale factor set to $s \approx 88.6$, which corresponds to a context size of 355k. They show that the network extrapolates up to 100k context without ever seeing those context sizes during training. Similar to 3.1 and Rozière et al. [31], YaRN also supports training with a higher scale factor s than the length of the dataset. Due to compute constraints, we test only $s = 32$ by further fine-tuning the $s = 16$ model for 200 steps using the same dataset with 64k context.

We show in 4.3.1 that the $s = 32$ model successfully extrapolates up to 128k context using only 64k context during training. Unlike previous "blind" interpolation methods, YaRN is much more efficient at transfer learning when increasing the scale s . This demonstrates successful transfer learning from $s = 16$ to $s = 32$ without the network needing to relearn the interpolated embeddings, as the $s = 32$ model is equivalent to the $s = 16$ model across the entire context size, despite only being trained on $s = 32$ for 200 steps.

4.3 Evaluation

The evaluations focus on three aspects:

1. the perplexity scores of fine-tuned models with extended context window,
2. the passkey retrieval task on fine-tuned models,
3. the common LLM benchmark results of fine-tuned models,

4.3.1 Long Sequence Language Modeling

To evaluate the long sequence language modeling performances, we use the GovReport [18] and Proof-pile [4] datasets both of which contain many long sequence samples. For all evaluations, the test splits of both datasets were used exclusively. All perplexity evaluations were calculated using the sliding window method from Press et al. [27] with $S = 256$.

Firstly, we evaluated how the model performed as the context window increased. We selected 10 random samples from Proof-pile with at least 128k tokens each and evaluated the perplexity of each of these samples when truncated at 2k steps from a sequence length of 2k tokens through 128k tokens.

Table 1 shows a side-by-side comparison of Llama-2 model extended from 4096 to 8192 context length via PI (LLongMA-2 7b⁵), "NTK-aware" and YaRN. Note that PI and "NTK-aware" models were trained using the methodology in Chen et al. [9], while YaRN used the same methodology but 2.5x less training steps and data, as described in 4.

⁵LLongMA-2 7b [28] is fine-tuned from Llama-2 7b, trained at 8k context length with PI using the RedPajama dataset [12].

在已建立的基准测试上评估所得模型，发现它们超越了所有其他上下文窗口扩展方法。

我们广泛遵循了 [9]中概述的训练和评估程序。

4.1 训练

预训练中，我们扩展了Llama 2 [39] 7B和13B参数模型。除了根据3.4中描述的 $s = 16$ 和 $s = 32$ 计算嵌入频率外，我们对LLaMA模型架构没有进行任何更改。

我们使用了学习率为 $\{2 \times 10^{-5}\}$ 、无权重衰减、20步线性预热，并配合AdamW [24] $\beta_1 = 0.9$ 和 $\beta_2 = 0.95$ 。对于 $s = 16$ ，我们使用PyTorch [26] 完全分片数据并行 [42] 和 Flash Attention 2 [13] 在PG19数据集 [29] 上进行微调，该数据集被分割为64k个段，并以 BOS和EOS标记开头和结尾。对于 $s = 32$ ，我们遵循了相同的过程，但从完成的 $s = 16$ 检查点开始，额外训练了200步。

4.2 外推和迁移学习

在Code Llama [31], 中，使用了一个上下文大小为16k的数据集，缩放因子设置为 $s \approx 88.6$ ，这对应于355k的上下文大小。他们表明，网络可以在训练过程中从未见过这些上下文大小的情况下，外推到100k上下文。类似于3.1和Rozière等人 [31],，YaRN也支持使用比数据集长度更高的缩放因子 s 进行训练。由于计算限制，我们仅通过使用64k上下文的数据集进一步微调 $s = 16$ 模型200步来测试 $s = 32$ 。

我们在4.3.1中展示了， $s = 32$ 模型仅使用64k上下文在训练期间成功外推到128k上下文。与之前的“盲目”插值方法不同，YaRN在增加缩放 s 时，迁移学习效率更高。这表明，网络无需重新学习插值嵌入，即可从 $s = 16$ 成功迁移到 $s = 32$ ，因为 $s = 32$ 模型在整个上下文大小范围内等同于 $s = 16$ 模型，尽管仅训练了 $s = 32$ 200步。

4.3 评估

评估聚焦于三个方面：

1. 扩展上下文窗口的微调模型的困惑度分数，
2. 微调模型上的密钥检索任务，
3. 微调模型的通用 LLM 基准结果，

4.3.1 长序列语言建模

为了评估长序列语言建模性能，我们使用了 GovReport [18] 和 Proof-pile [4] 数据集，这两个数据集都包含许多长序列样本。在所有评估中，我们仅使用了这两个数据集的测试集。所有困惑度评估都是使用来自 Press 等人 [27] 的滑动窗口方法，并使用 $S = 256$ 计算得出的。

首先，我们评估了模型在上下文窗口增加时的表现。我们从 Proof-pile 中随机选择了 10 个样本，每个样本至少包含 128k 个标记，并评估了这些样本在从 2k 个标记的序列长度到 128k 个标记的序列长度，每次截断 2k 步时的困惑度。

表1展示了通过PI（LLongMA-2 7bs）将Llama-2模型从4096扩展到8192上下文长度的对比，包括"NTK-aware"和YaRN。请注意，PI和"NTK-aware"模型使用陈等人 [9], 的方法进行训练，而YaRN使用相同的方法，但训练步数和数据减少了2.5倍，如4所述。

^sLLongMA-2 7b [28]是从 Llama-2 7b 微调而来，使用 RedPajama 数据集 [12]在 8k 上下文长度和 PI 下进行训练。

Extension Method	Trained Tokens	Context Window	Evaluation Context Window Size				
			2048	4096	6144	8192	10240
PI ($s = 2$)	1B	8k	3.92	3.51	3.51	3.34	8.07
NTK ($\theta = 20k$)	1B	8k	4.20	3.75	3.74	3.59	6.24
YaRN ($s = 2$)	400M	8k	3.91	3.50	3.51	3.35	6.04

Table 1: Sliding window perplexity ($S = 256$) of ten 128k Proof-pile documents over Llama-2 extended via PI, NTK and YaRN

We further evaluated YaRN at the scale factor $s = 16, 32$ and compared them against a few open-source models fine-tuned from Llama-2 and extended to more than 32k context window such as Together.ai [37] and "NTK-aware" Code Llama [31]. The results are summarized in Table 2 (with a more detailed plot in Figure 1).

Model Size	Model Name	Context Window	Extension Method	Evaluation Context Window Size				
				8192	32768	65536	98304	131072
7B	Together	32k	PI	3.50	2.64	$> 10^2$	$> 10^3$	$> 10^4$
7B	Code Llama	100k	NTK	3.71	2.74	2.55	2.54	2.71
7B	YaRN ($s = 16$)	64k	YaRN	3.51	2.65	2.42	$> 10^1$	$> 10^1$
7B	YaRN ($s = 32$)	128k	YaRN	3.56	2.70	2.45	2.36	2.37
13B	Code Llama	100k	NTK	3.54	2.63	2.41	2.37	2.54
13B	YaRN ($s = 16$)	64k	YaRN	3.25	2.50	2.29	$> 10^1$	$> 10^1$
13B	YaRN ($s = 32$)	128k	YaRN	3.29	2.53	2.31	2.23	2.24

Table 2: Sliding window perplexity ($S = 256$) of ten 128k Proof-pile documents truncated to evaluation context window size

We observe that the model exhibits strong performance across the entire targeted context size, with YaRN interpolation being the first method to successfully extend the effective context size of Llama 2 to 128k. Of particular note are the YaRN ($s = 32$) models, which show continued declining perplexity through 128k, despite the fine-tuning data being limited to 64k tokens in length, demonstrating that the model is able to generalize to unseen context lengths.

Furthermore, in Appendix B.1, we show the results of the average perplexity on 50 untruncated GovReport documents with at least 16k tokens per sample evaluated on the setting of 32k maximal context window without Dynamic Scaling in Table 4. Similar to the Proof-pile results, the GovReport results show that fine-tuning with YaRN achieves good performance on long sequences.

4.3.2 Passkey Retrieval

The passkey retrieval task as defined in [25] measures a model’s ability to retrieve a simple passkey (i.e., a five-digit number) from amongst a large amount of otherwise meaningless text. For our evaluation of the models, we performed 10 iterations of the passkey retrieval task with the passkey placed at a random location uniformly distributed across the evaluation context window on different context window sizes ranging from 8k to 128k. Both 7b and 13b models fine-tuned using YaRN at 128k context size passes the passkey retrieval task with very high accuracy ($> 99\%$) within the entire context window size. We show detailed results in Appendix B.2.

4.3.3 Standardized Benchmarks

The Hugging Face Open LLM Leaderboard [19] compares a multitude of LLMs across a standardized set of four public benchmarks. Specifically, we use 25-shot ARC-Challenge [11], 10-shot HellaSwag [41], 5-shot MMLU [17], and 0-shot TruthfulQA [23].

To test the degradation of model performance under context extension, we evaluated our models using this suite and compared it to established scores for the Llama 2 baselines as well as publicly available PI and "NTK-aware" models. The results are summarized in Table 3.

扩展方法	训练标记	上下文窗口	评估上下文窗口大小				
			2048	4096	6144	8192	10240
PI ($s = 2$)	1B	8k	3.92	3.51	3.51	3.34	8.07
NTK ($\theta = 20k$)	1B	8k	4.20	3.75	3.74	3.59	6.24
YaRN ($s = 2$)	400M	8k	3.91	3.50	3.51	3.35	6.04

表1: 滑动窗口困惑度 ($S = 256$), 十份128k证明堆文档在Llama-2 extended上通过PI、NTK和YaRN

我们进一步在缩放因子 $s = 16, 32$ 的规模下评估了YaRN, 并将它们与从Llama-2微调而来、扩展到超过32k上下文窗口的几个开源模型进行了比较, 例如Together.ai [37] 和 "NTK-aware" CodeLlama [31]。结果总结在表2中 (图1中有更详细的图表)。

模型Size	模型Name	上下文窗口	扩展方法	评估上下文窗口大小				
				8192	32768	65536	98304	131072
7B	一起	32k	PI	3.50	2.64	$> 10^2$	$> 10^3$	$> 10^4$
7B	Code Llama	100k	NTK	3.71	2.74	2.55	2.54	2.71
7B	YaRN ($s = 16$)	64k	YaRN	3.51	2.65	2.42	$> 10^1$	$> 10^1$
7B	YaRN ($s = 32$)	128k	YaRN	3.56	2.70	2.45	2.36	2.37
13B	Code Llama	100k	NTK	3.54	2.63	2.41	2.37	2.54
13B	YaRN ($s = 16$)	64k	YaRN	3.25	2.50	2.29	$> 10^1$	$> 10^1$
13B	YaRN ($s = 32$)	128k	YaRN	3.29	2.53	2.31	2.23	2.24

表2: 十个128k证明堆文档的滑动窗口困惑度 ($S = 256$), 截断至评估上下文窗口大小

我们观察到模型在整个目标上下文大小范围内表现出色, YaRN插值是第一个成功将Llama 2的有效上下文大小扩展到128k的方法。特别值得注意的是YaRN ($s = 32$) 模型, 尽管微调数据长度仅限于64k标记, 但该模型在128k范围内持续下降的困惑度, 表明模型能够泛化到未见过的上下文长度。

此外, 在附录B.1中, 我们展示了在50个未截断的GovReport文档上 (每个样本至少包含16k个标记) 的平均困惑度结果, 这些文档在表4的32k最大上下文窗口设置下 (无动态缩放) 进行评估。与证明堆结果类似, GovReport结果表明, 使用YaRN进行微调在长序列上取得了良好的性能。

4.3.2 密钥检索

如 [25] 中定义的密钥检索任务, 衡量模型从大量无意义文本中检索简单密钥 (即五位数字) 的能力。在我们的模型评估中, 我们在不同上下文窗口大小 (从8k到128k) 上对密钥检索任务进行了10次迭代, 密钥在评估上下文窗口中随机位置均匀分布。7b和13b模型在128k上下文大小下使用YaRN进行微调, 在整个上下文窗口大小范围内以非常高的准确率 ($> 99\%$) 通过了密钥检索任务。我们在附录B.2中展示了详细结果。

4.3.3 标准化基准测试

HuggingFace公开LLM排行榜 [19] 在一组标准化的四个公共基准测试中比较了多种大型语言模型。具体来说, 我们使用25-shot ARC-Challenge [11], 10-shot Hellaswag [41], 5-shot MMLU [17], 和0-shot TruthfulQA [23]。

为了测试模型在上下文扩展下的性能退化, 我们使用该套件评估了我们的模型, 并将其与Llama 2基线的已建立分数以及公开可用的PI和"NTK-aware"模型进行了比较。结果总结在表3中。

Model Size	Model Name	Context Window	Extension Method	ARC-c	Hellaswag	MMLU	TruthfulQA
7B	Llama 2	4k	None	53.1	77.8	43.8	39.0
7B	Together	32k	PI	47.6	76.1	43.3	39.2
7B	Code Llama	100k	NTK	39.9	60.8	31.1	37.8
7B	YaRN ($s = 16$)	64k	YaRN	52.3	78.8	42.5	38.2
7B	YaRN ($s = 32$)	128k	YaRN	52.1	78.4	41.7	37.3
13B	Llama 2	4k	None	59.4	82.1	55.8	37.4
13B	Code Llama	100k	NTK	40.9	63.4	32.8	43.8
13B	YaRN ($s = 16$)	64k	YaRN	58.1	82.3	52.8	37.8
13B	YaRN ($s = 32$)	128k	YaRN	58.0	82.2	51.9	37.3

Table 3: Performance of context window extensions methods on the Hugging Face Open LLM benchmark suite compared with original Llama 2 baselines

We observe that there is minimal performance degradation between the YaRN models and their respective Llama 2 baselines. We also observe that there was on average a 0.49% drop in scores between the YaRN $s = 16$ and $s = 32$ models. From this we conclude that the the iterative extension from 64k to 128k results in negligible performance loss.

5 Conclusion

In conclusion, we have shown that YaRN improves upon all existing RoPE interpolation methods and can act as a drop-in replacement to PI, with no downsides and minimal implementation effort. The fine-tuned models preserve their original abilities on multiple benchmarks while being able to attend to a very large context size. Furthermore, YaRN allows efficient extrapolation with fine-tuning on shorter datasets and can take advantage of transfer learning for faster convergence, both of which are crucial under compute-constrained scenarios. Finally, we have shown the effectiveness of extrapolation with YaRN where it is able to "train short, and test long".

模型 Size	模型 Name	上下文窗口	扩展方法	ARC-c	Hellaswag	MMLU	TruthfulQA
7B	Llama 2	4k	None	53.1	77.8	43.8	39.0
7B	一起	32k	PI	47.6	76.1	43.3	39.2
7B	Code Llama	100k	NTK	39.9	60.8	31.1	37.8
7B	YaRN ($s = 16$)	64k	YaRN	52.3	78.8	42.5	38.2
7B	YaRN ($s = 32$)	128k	YaRN	52.1	78.4	41.7	37.3
13B	Llama 2	4k	None	59.4	82.1	55.8	37.4
13B	Code Llama	100k	NTK	40.9	63.4	32.8	43.8
13B	YaRN ($s = 16$)	64k	YaRN	58.1	82.3	52.8	37.8
13B	YaRN ($s = 32$)	128k	YaRN	58.0	82.2	51.9	37.3

表3: 在Hugging Face Open LLM基准套件上比较上下文窗口扩展方法对Llama 2原始基线的性能

我们观察到，YaRN模型与其各自的Llama 2基线之间性能退化很小。我们还观察到，YaRN $s = 16$ 和 $s = 32$ 模型之间的分数平均下降了0.49%。由此我们得出结论，从64k到128k的迭代扩展导致性能损失可以忽略不计。

5 结论

总之，我们证明了YaRN在所有现有的RoPE插值方法上都有所改进，并且可以作为PI的直接替代品，没有任何缺点和最小的实现工作量。微调模型在多个基准测试中保留了它们原有的能力，同时能够处理非常大的上下文大小。此外，YaRN允许在较短的 数据集上进行微调以实现高效的外推，并且可以利用迁移学习来加快收敛速度，这两点在计算受限的情况下都至关重要。最后，我们展示了YaRN外推的有效性，它能够实现“训练短，测试长”。

6 Reproducibility

To aid in reproducibility, we provide, as supplementary material, the entirety of the code used to train the YaRN models in Table 2, as well as the evaluation code that produced Figure 1 and Tables 1, 2, 3, 4, and 5. The code also contains implementations of various extension methods referenced throughout the paper. For training YaRN, we used the publicly available PG19 dataset [29] tokenized to 64k tokens.

References

- [1] MistralLite. URL <https://huggingface.co/amazon/MistralLite>.
- [2] Introducing Qwen-7B: Open foundation and human-aligned models (of the state-of-the-arts). URL https://github.com/QwenLM/Qwen-7B/blob/main/tech_memo.md.
- [3] Long-data collections. URL <https://huggingface.co/datasets/togethercomputer/Long-Data-Collections>.
- [4] Z. Azerbayev, E. Ayers, , and B. Piotrowski. Proof-pile, 2022. URL <https://github.com/zhangir-azerbayev/proof-pile>.
- [5] S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonell, J. Phang, M. Pieler, U. S. Prashanth, S. Purohit, L. Reynolds, J. Tow, B. Wang, and S. Weinbach. GPT-NeoX-20B: An open-source autoregressive language model, 2022. arXiv: 2204.06745.
- [6] bloc97. NTK-Aware Scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation., 2023. URL https://www.reddit.com/r/LocalLLaMA/comments/141z7j5/ntkaware_scaled_rope_allows_llama_models_to_have/.
- [7] bloc97. Add NTK-Aware interpolation "by parts" correction, 2023. URL <https://github.com/jquesnelle/scaled-rope/pull/1>.
- [8] C. Chen. Transformer Inference Arithmetic, 2022. URL <https://kipp.ly/blog/transformer-inference-arithmetic/>.
- [9] S. Chen, S. Wong, L. Chen, and Y. Tian. Extending context window of large language models via positional interpolation, 2023. arXiv: 2306.15595.
- [10] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel. PaLM: Scaling language modeling with pathways, 2022. arXiv: 2204.02311.
- [11] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try ARC, the AI2 Reasoning Challenge, 2018. arXiv: 1803.05457.
- [12] T. Computer. Redpajama: An open source recipe to reproduce llama training dataset, 2023. URL <https://github.com/togethercomputer/RedPajama-Data>.
- [13] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. arXiv: 2307.08691.

6 可重复性

为了提高可重复性，我们作为补充材料提供了用于训练表2中YaRN模型的全部代码，以及生成图1和表1、表2、表3、表4和表5的评估代码。该代码还包含了论文中引用的各种扩展方法的实现。为了训练YaRN，我们使用了公开可用的PG19数据集 [29]分词到64k个标记。

参考文献

- [1] MistralLite. URL <https://huggingface.co/amazon/MistralLite>.
- [2] Introducing Qwen-7B: Open foundation and human-aligned models (of the state-of-the-arts). URL https://github.com/QwenLM/Qwen-7B/blob/main/tech_memo.md.
- [3] Long-data collections. URL <https://huggingface.co/datasets/togethercomputer/Long-Data-Collections>.
- [4] Z. Azerbayev, E. Ayers, , and B. Piotrowski. Proof-pile, 2022. URL <https://github.com/zhangir-azerbayev/proof-pile>.
- [5] S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonell, J. Phang, M. Pieler, U. S. Prashanth, S. Purohit, L. Reynolds, J. Tow, B. Wang, and S. Weinbach. GPT-NeoX-20B: An open-source autoregressive language model, 2022. arXiv: 2204.06745.
- [6] bloc97. NTK-Aware Scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation., 2023. URL https://www.reddit.com/r/LocalLLaMA/comments/141z7j5/ntkaware_scaled_rope_allows_llama_models_to_have/.
- [7] bloc97. Add NTK-Aware interpolation "by parts" correction, 2023. URL <https://github.com/jquesnelle/scaled-rope/pull/1>.
- [8] C. Chen. Transformer Inference Arithmetic, 2022. URL <https://kipp.ly/blog/transformer-inference-arithmetic/>.
- [9] S. Chen, S. Wong, L. Chen, and Y. Tian. Extending context window of large language models via positional interpolation, 2023. arXiv: 2306.15595.
- [10] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel. PaLM: Scaling language modeling with pathways, 2022. arXiv: 2204.02311.
- [11] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try ARC, the AI2 Reasoning Challenge, 2018. arXiv: 1803.05457.
- [12] T. Computer. Redpajama: An open source recipe to reproduce llama training dataset, 2023. URL <https://github.com/togethercomputer/RedPajama-Data>.
- [13] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. arXiv: 2307.08691.

- [14] emozilla. Dynamically Scaled RoPE further increases performance of long context LLaMA with zero fine-tuning, 2023. URL https://www.reddit.com/r/LocalLLaMA/comments/14mrgpr/dynamically_scaled_rope_further_increases/.
- [15] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin. Convolutional sequence to sequence learning, 2017. arXiv: 1705.03122.
- [16] C. Han, Q. Wang, W. Xiong, Y. Chen, H. Ji, and S. Wang. LM-Infinite: Simple on-the-fly length generalization for large language models, 2023. arXiv: 2308.16137.
- [17] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [18] L. Huang, S. Cao, N. Parulian, H. Ji, and L. Wang. Efficient attentions for long document summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1419–1436. Association for Computational Linguistics, June 2021.
- [19] Hugging Face. Open LLM Leaderboard, 2023. URL https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard.
- [20] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. Mistral 7b, 2023.
- [21] kaioke dev. Things I’m learning while training superhot., 2023. URL <https://kaioke dev.github.io/til#extending-context-to-8k>.
- [22] A. Kazemnejad, I. Padhi, K. N. Ramamurthy, P. Das, and S. Reddy. The impact of positional encoding on length generalization in transformers, 2023. arXiv: 2305.19466.
- [23] S. Lin, J. Hilton, and O. Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, May 2022.
- [24] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [25] A. Mohtashami and M. Jaggi. Landmark attention: Random-access infinite context length for transformers, 2023. arXiv: 2305.16300.
- [26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimsheine, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, pages 8024–8035, 2019.
- [27] O. Press, N. Smith, and M. Lewis. Train Short, Test Long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.
- [28] J. Quesnelle, E. Shippole, and "Kaioke dev". Llongma: Scaling rotary embeddings through linear positional interpolation. <https://huggingface.co/conceptofmind/LLongMA-2-7b/>, 2023.
- [29] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- [30] A. Roberts, C. Raffel, K. Lee, M. Matena, N. Shazeer, P. J. Liu, S. Narang, W. Li, and Y. Zhou. Exploring the limits of transfer learning with a unified text-to-text transformer. Technical report, Google, 2019.

- [14] emozilla. Dynamically Scaled RoPE further increases performance of long context LLaMA with zero fine-tuning, 2023. URL https://www.reddit.com/r/LocalLLaMA/comments/14mrgpr/dynamically_scaled_rope_further_increases/.
- [15] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin. Convolutional sequence to sequence learning, 2017. arXiv: 1705.03122.
- [16] C. Han, Q. Wang, W. Xiong, Y. Chen, H. Ji, and S. Wang. LM-Infinite: Simple on-the-fly length generalization for large language models, 2023. arXiv: 2308.16137.
- [17] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [18] L. Huang, S. Cao, N. Parulian, H. Ji, and L. Wang. Efficient attentions for long document summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1419–1436. Association for Computational Linguistics, June 2021.
- [19] Hugging Face. Open LLM Leaderboard, 2023. URL https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard.
- [20] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. Mistral 7b, 2023.
- [21] kaioke dev. Things I’m learning while training superhot., 2023. URL <https://kaioke dev.github.io/til#extending-context-to-8k>.
- [22] A. Kazemnejad, I. Padhi, K. N. Ramamurthy, P. Das, and S. Reddy. The impact of positional encoding on length generalization in transformers, 2023. arXiv: 2305.19466.
- [23] S. Lin, J. Hilton, and O. Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, May 2022.
- [24] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [25] A. Mohtashami and M. Jaggi. Landmark attention: Random-access infinite context length for transformers, 2023. arXiv: 2305.16300.
- [26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimsheine, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, pages 8024–8035, 2019.
- [27] O. Press, N. Smith, and M. Lewis. Train Short, Test Long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.
- [28] J. Quesnelle, E. Shippole, and "Kaioke dev". Llongma: Scaling rotary embeddings through linear positional interpolation. <https://huggingface.co/conceptofmind/LLongMA-2-7b/>, 2023.
- [29] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- [30] A. Roberts, C. Raffel, K. Lee, M. Matena, N. Shazeer, P. J. Liu, S. Narang, W. Li, and Y. Zhou. Exploring the limits of transfer learning with a unified text-to-text transformer. Technical report, Google, 2019.

- [31] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve. Code Llama: Open foundation models for code, 2023. arXiv: 2308.12950.
- [32] P. Shaw, J. Uszkoreit, and A. Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 464–468, New Orleans, Louisiana, June 2018. Association for Computational Linguistics.
- [33] J. Su. Rectified rotary position embeddings. <https://github.com/bojone/rerope>, 2023.
- [34] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding, 2022. arXiv: 2104.09864.
- [35] Y. Sun, L. Dong, B. Patra, S. Ma, S. Huang, A. Benhaim, V. Chaudhary, X. Song, and F. Wei. A length-extrapolatable transformer, 2022. arXiv: 2212.10554.
- [36] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS’20*, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- [37] Together.ai. LLaMA-2-7B-32K, 2023. URL <https://huggingface.co/togethercomputer/LLaMA-2-7B-32K>.
- [38] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. LLaMA: Open and efficient foundation language models, 2023. arXiv: 2302.13971.
- [39] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [41] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [42] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, B. Nguyen, G. Chauhan, Y. Hao, and S. Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel, 2023. arXiv: 2304.11277.

- [31] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve. Code Llama: Open foundation models for code, 2023. arXiv: 2308.12950.
- [32] P. Shaw, J. Uszkoreit, and A. Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 464–468, New Orleans, Louisiana, June 2018. Association for Computational Linguistics.
- [33] J. Su. Rectified rotary position embeddings. <https://github.com/bojone/rerope>, 2023.
- [34] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding, 2022. arXiv: 2104.09864.
- [35] Y. Sun, L. Dong, B. Patra, S. Ma, S. Huang, A. Benhaim, V. Chaudhary, X. Song, and F. Wei. A length-extrapolatable transformer, 2022. arXiv: 2212.10554.
- [36] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS’20*, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- [37] Together.ai. LLaMA-2-7B-32K, 2023. URL <https://huggingface.co/togethercomputer/LLaMA-2-7B-32K>.
- [38] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. LLaMA: Open and efficient foundation language models, 2023. arXiv: 2302.13971.
- [39] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [41] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [42] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, B. Nguyen, G. Chauhan, Y. Hao, and S. Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel, 2023. arXiv: 2304.11277.

A Additional details on interpolation methods

A.1 Short notes on the deduction of "NTK-aware" interpolation

In Section 3.1, we introduce a change of basis from b to b' in the definition of "NTK-aware" interpolation method. Here is a short note on its mathematical deduction.

Recall that our goal is to spread out the interpolation pressure across the hidden dimensions using a base-change instead of scaling the frequencies by a fixed factor s . The property we want to guarantee is that: The lowest frequency needs to be scaled as much as linear positional scaling and the highest frequency to stay constant.

We introduce a new base b' such that the last dimension matches the wavelength of linear interpolation with a scale factor s . Since the original RoPE method skips odd dimensions in order to concatenate both $\cos(\frac{2\pi x}{\lambda})$ and $\sin(\frac{2\pi x}{\lambda})$ components into a single embedding, the last dimension $d \in D$ is $|D| - 2$.

The new base b' can be chosen so that

$$b'^{\frac{|D|-2}{|D|}} = s \cdot b^{\frac{|D|-2}{|D|}}. \quad (23)$$

Solving for b' yields

$$b' = b \cdot s^{\frac{|D|}{|D|-2}}. \quad (24)$$

A.2 The impact of pre-softmax scaling of YaRN on perplexity

In Section 3.4, we mention the impact of the factor t inside the softmax computation of attention weights. Here we fix 896 16k-token documents from RedPajama [12]⁶, and calculate their perplexity scores with different scaling $1/\sqrt{t}$. The result is in Figure 2. For comparison, recall that our recommended factor in this case ($s = 8$) is given by the following.

$$\sqrt{\frac{1}{t}} = 0.1 \ln(s) + 1 \approx 1.208. \quad (25)$$

⁶We choose RedPajama because it is the open-source dataset closest to the training dataset of LLaMA as far as we are aware of.

关于插值方法的补充细节

A.1 关于“NTK-aware”插值推导的简短说明

在3.1节中，我们在“NTK-aware”插值方法的定义中引入了从 b 到 b' 的基变换。以下是关于其数学推导的简短说明。

回想一下，我们的目标是通过基变换而不是通过固定因子 s 缩放频率来将插值压力分布在隐藏维度上。我们希望保证的性质是：最低频率需要像线性位置缩放一样进行缩放，而最高频率需要保持不变。

我们引入一个新的基 b' ，使得最后一个维度与线性插值的波长匹配，并具有缩放因子 s 。由于原始的RoPE方法为了将 $\cos(\frac{2\pi x}{\lambda})$ 和 $\sin(\frac{2\pi x}{\lambda})$ 组件连接成一个嵌入而跳过奇数维度，最后一个维度 $d \in D$ 是 $|D| - 2$ 。

新的基 b' 可以选择使得

$$b'^{\frac{|D|-2}{|D|}} = s \cdot b^{\frac{|D|-2}{|D|}}. \quad (23)$$

求解 b' 得到

$$b' = b \cdot s^{\frac{|D|}{|D|-2}}. \quad (24)$$

A.2 YaRN的softmax前缩放对困惑度的影响

在3.4节中，我们提到了因子 t 对注意力权重softmax计算的影响。在这里，我们固定了来自 RedPajama [12]⁶的896个16k-token文档，并使用不同的缩放 $1/\sqrt{t}$ 计算它们的困惑度分数。结果如图2所示。为了比较，请回忆在这种情况下 ($s = 8$) 我们推荐的因子由以下公式给出。

$$\sqrt{\frac{1}{t}} = 0.1 \ln(s) + 1 \approx 1.208. \quad (25)$$

⁶我们选择RedPajama，因为它是我们所知与LLaMA训练数据集最接近的开源数据集。

To show the impact of the factor $1/\sqrt{t}$ on different token positions, we cut each 16k-token document into chunks of 2048 tokens, and further plot the mean perplexity change comparing to $t = 1$ in percentages

$$\frac{\text{ppl}(t) - \text{ppl}(t = 1)}{\text{ppl}(t = 1)} \quad (26)$$

of each chunk. The plot is shown in Figure 3.

To further demonstrate the best values of t across all samples over different token positions, we plot the sample counts with minimal perplexity at a given $1/\sqrt{t}$ for each of the 8 position segments over the 16k-token range in Figure 4.

We observe that:

- for a suitable t , a sample may obtain better perplexity scores across the extended context window;
- the best value of t is mostly consistent across different samples and different positions.

We remark that this finding is consistent for different values of s and the best value of t follows our recommended formula (Eq. 22) closely.

B Additional tables and charts

B.1 GovReport evaluations

In Section 4.3.1, we mention the evaluation on GovReport documents. The evaluation results are detailed in Table 4 below.

B.2 Passkey Retrieval

Here we can observe that the lowest perplexity point alone does not provide a comprehensive depiction on the "effective context size" that an LLM can attend to. While the Code Llama 13b model exhibits increasing perplexity above 100k context lengths, it was still able to accurately retrieve the passkey at a context length of 128k. This suggest that while the output of Code Llama might start to degrade in quality above 100k context size, it is still able to maintain strong retrieval capabilities.

In addition, as YaRN with $s = 32$ was trained for 200 more steps than YaRN with $s = 16$ while having a higher passkey accuracy with similar perplexity, we hypothesize that perplexity may not be a great indicator of whether an LLM is able to attend to all tokens and does not exhaustively determine long context performance. This also suggests that the YaRN models with $s = 16$ might be relatively undertrained for the passkey retrieval task.

B.3 Dynamic scaling on models without any fine-tuning

We first recall from Section 3.3 that the Dynamic Scaling technique is an inference-time technique that dynamically update the factor s in interpolation methods such as PI, "NTK-by-parts" and YaRN. We choose the original Llama 2, fix a sample in GovReport and calculate its perplexity on a sliding window of 256 tokens using RoPE, Dynamic-PI and Dynamic-YaRN. Since the original maximal context length of Llama 2 is 4096, we observe that Dynamic Scaling effectively extend the inference length and Dynamic-YaRN achieves better performance than Dynamic-PI. The resulting chart is in Figure 5.

We see that

- Dynamic Scaling effectively prevents the blow-up of perplexity score beyond pretrained context window;
- Dynamic-YaRN outperforms Dynamic-PI in terms of long-range perplexity on pretrained Llama-2 without any finetuning.

为了展示因子 $1/\sqrt{t}$ 对不同token位置的影响，我们将每个16k-token文档切割成2048个token的块，并进一步绘制与 $t = 1$ 相比的平均困惑度变化百分比

$$\frac{\text{ppl}(t) - \text{ppl}(t = 1)}{\text{ppl}(t = 1)} \quad (26)$$

每个块。图表显示在图3中。

为了进一步展示在不同token位置上所有样本中 t 的最佳值，我们在图4中绘制了在16k标记范围内每个8个位置段上给定 $1/\sqrt{t}$ 的最小困惑度下的样本计数。

我们观察到：

- 对于合适的 t ，一个样本可能在扩展上下文窗口中获得更好的困惑度分数；
- t 的最佳值在不同样本和不同位置上基本一致。

我们指出，这一发现适用于不同的 s 值，并且 t 的最佳值与我们的推荐公式（公式22）非常接近。

B 补充表格和图表

B.1 GovReport评估

在4.3.1节中，我们提到了对GovReport文档的评估。评估结果详见下表4。

B.2 密钥检索

在这里我们可以观察到，仅凭最低困惑度点并不能全面描述LLM能够关注的“有效上下文大小”。虽然Code Llama 13b模型在100k上下文长度以上表现出困惑度增加，但它仍然能够在128k上下文长度下准确检索密钥。这表明，虽然Code Llama在100k上下文大小以上的输出质量可能开始下降，但它仍然能够保持强大的检索能力。

此外，由于YaRN with $s = 32$ 比YaRN with $s = 16$ 多训练了200步，并且具有相似的困惑度但更高的密钥准确率，我们假设困惑度可能不是衡量LLM是否能够关注所有标记的很好指标，也不能完全决定长上下文性能。这也表明，具有 $s = 16$ 的YaRN模型可能相对未充分训练以执行密钥检索任务。

B.3 对未经微调的模型进行动态缩放

我们首先从第3.3节中回忆起，动态缩放技术是一种推理时技术，它动态更新插值方法（如PI、"NTK-by-parts"和YaRN）中的因子 s 。我们选择原始Llama2，在GovReport中固定一个样本，并使用RoPE、动态PI和动态YaRN在256个标记的滑动窗口上计算其困惑度。由于Llama 2的原始最大上下文长度为4096，我们观察到动态缩放有效地扩展了推理长度，并且动态YaRN比动态PI表现更好。结果图表如图5所示。

我们看到

- 动态缩放有效地防止了困惑度分数在预训练上下文窗口之外激增；
- 在未经任何微调的预训练Llama-2上，动态YaRN在长距离困惑度方面优于动态PI。

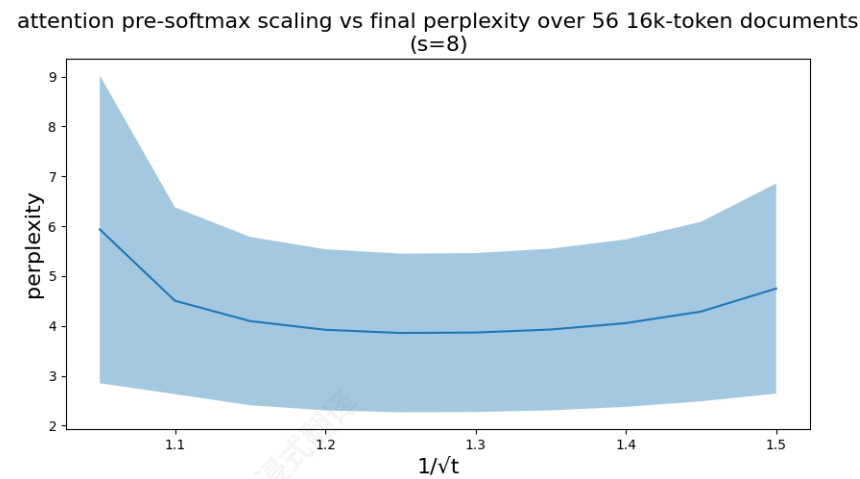


Figure 2: Fix $s = 8$, compare the LLaMA 7b perplexity on 896 16k-token documents over different scaling $1/\sqrt{t}$. The shaded area represents 1 standard deviation (68%).

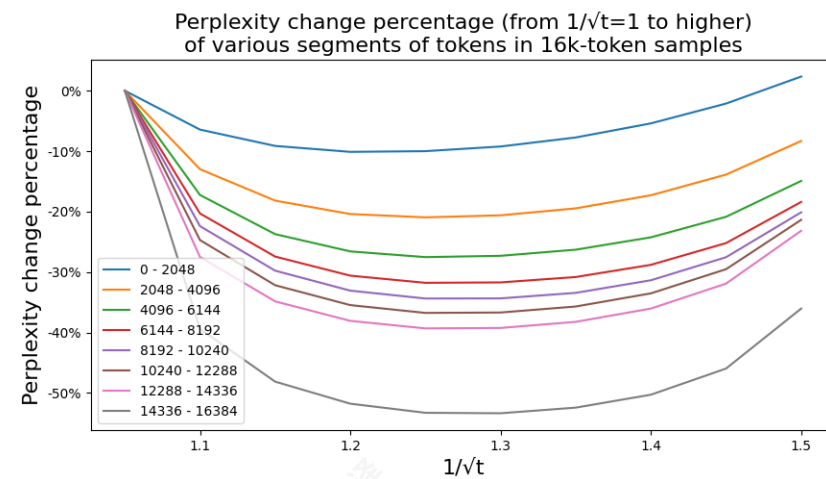


Figure 3: Fix $s = 8$, compare the mean of perplexity change percentages $\frac{ppl(t) - ppl(t=1)}{ppl(t=1)}$ at different segments of token positions on 896 16k-token documents over different scaling $1/\sqrt{t}$.

Model Size	Model Name	Context Window	Extension Method	Perplexity
7B	Together	32k	PI	3.67
7B	Code Llama	100k	NTK	4.44
7B	YaRN ($s = 16$)	64k	YaRN	3.59
7B	YaRN ($s = 32$)	128k	YaRN	3.64
13B	Code Llama	100k	NTK	4.22
13B	YaRN ($s = 16$)	64k	YaRN	3.35
13B	YaRN ($s = 32$)	128k	YaRN	3.39

Table 4: Sliding window perplexity ($S = 256$) of 50 long GovReport documents with a fixed context window size of 32k

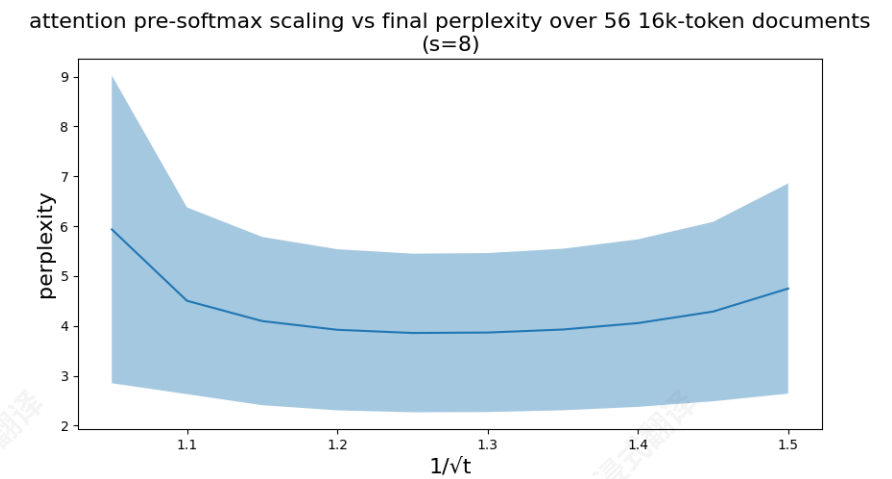


图2: 修复 $s = 8$, 比较在不同缩放 $1/\sqrt{t}$ 下 896 个 16k-token 文档的 LLaMA 7b 困惑度。阴影区域表示 1 个标准差 (68%)。

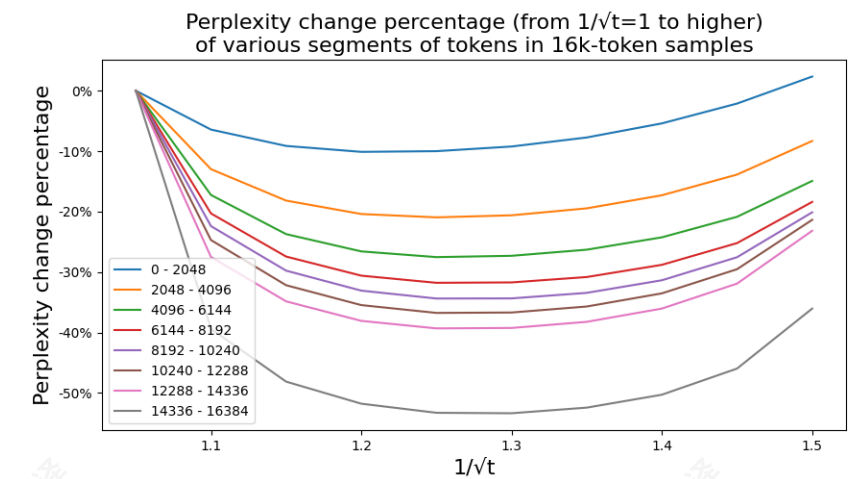


图3: 修复 $s = 8$, 比较在不同缩放 $1/t$ 下 896 个 16k-token 文档中不同 token 位置段的困惑度变化百分比的平均值 $\frac{ppl(t) - ppl(t=1)}{ppl(t=1)}$ 。

模型 Size	模型 Name	上下文窗口	扩展方法	困惑度
7B	一起	32k	PI	3.67
7B	Code Llama	100k	NTK	4.44
7B	YaRN ($s = 16$)	64k	YaRN	3.59
7B	YaRN ($s = 32$)	128k	YaRN	3.64
13B	Code Llama	100k	NTK	4.22
13B	YaRN ($s = 16$)	64k	YaRN	3.35
13B	YaRN ($s = 32$)	128k	YaRN	3.39

表4: 50长GovReport文档的滑动窗口困惑度($S = 256$), 固定上下文窗口大小为32k

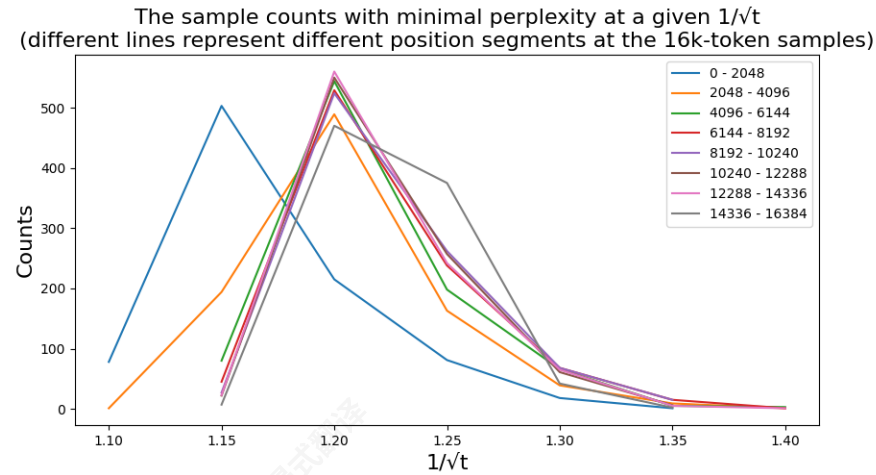


Figure 4: The sample counts (out of the 896 samples) with minimal perplexity at a given $1/\sqrt{t}$ for a given segment of token positions over the 16k-token range.

Model Size	Model Name	Scaling Factor (s)	Context Window	Training Data Context	Extension Method	Passkey Context	Passkey Accuracy
7B	Together	4	32k	32k	PI	32k	100%
7B	Code Llama	88.6	100k	16k	NTK	112k	94.3%
7B	YaRN	16	64k	64k	YaRN	64k	96.3%
7B	YaRN	32	128k	64k	YaRN	128k	99.4%
13B	Code Llama	88.6	100k	16k	NTK	128k	99.4%
13B	YaRN	16	64k	64k	YaRN	64k	97.5%
13B	YaRN	32	128k	64k	YaRN	128k	99.4%

Table 5: Passkey retrieval performance of various models. The passkey context denotes the maximum tested context window size where the accuracy of passkey retrieval was $\geq 80\%$, and the passkey accuracy is the average accuracy of passkey retrieval on all context sizes tested that were smaller or equal than the passkey context size.

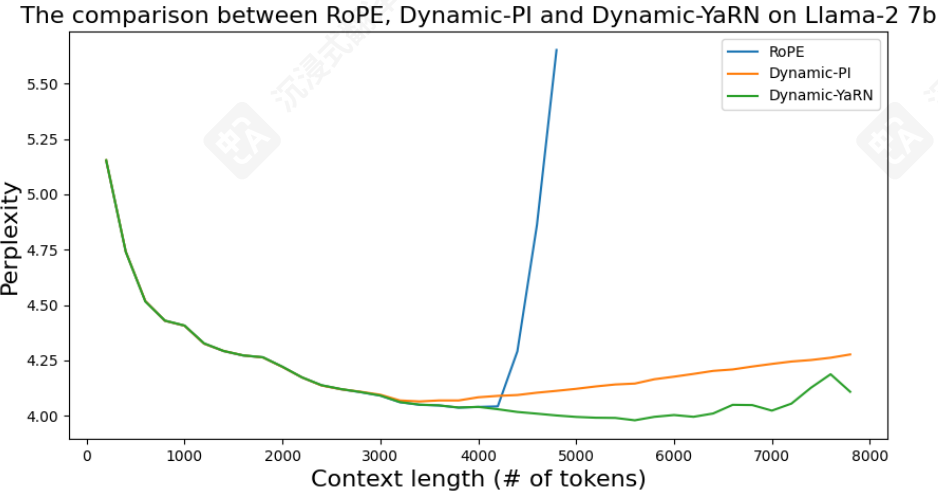


Figure 5: The comparison between RoPE, Dynamic-PI and Dynamic-YaRN using Llama 2 on a long GovReport sample. This model has not been finetuned for long context.

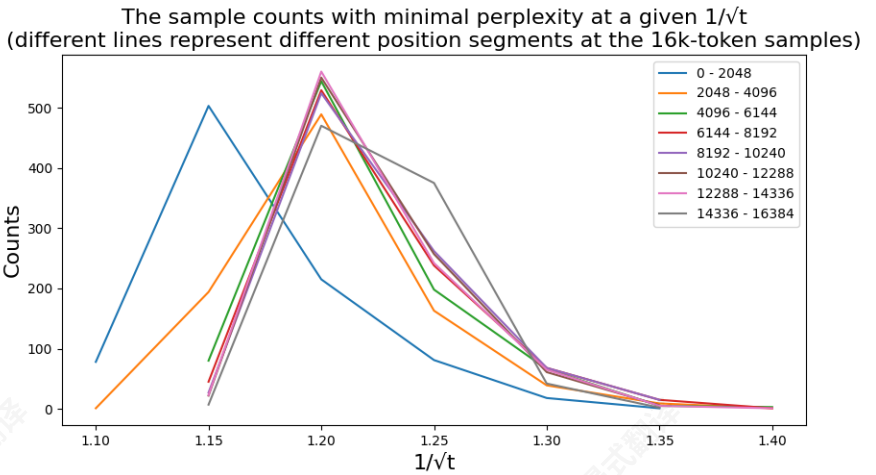


图4: 在16k标记范围内, 给定标记位置段在给定{s}时的样本计数 (在896个样本中) 具有最小困惑度。

模型 Size	模型 Name	缩放 因素 (s)	上下文 窗口	训练 数据上下文	扩展 方法	密钥 上下文	密钥 准确率
7B	一起	4	32k	32k	PI	32k	100%
7B	Code Llama	88.6	100k	16k	NTK	112k	94.3%
7B	YaRN	16	64k	64k	YaRN	64k	96.3%
7B	YaRN	32	128k	64k	YaRN	128k	99.4%
13B	Code Llama	88.6	100k	16k	NTK	128k	99.4%
13B	YaRN	16	64k	64k	YaRN	64k	97.5%
13B	YaRN	32	128k	64k	YaRN	128k	99.4%

表5: 各种模型的密钥检索性能。密钥上下文denotesthemaximumtested上下文窗口大小, 其中密钥检索的准确率为 $\geq 80\%$, 而密钥准确率是在所有测试的上下文大小中, 小于或等于密钥上下文大小的密钥检索平均准确率。

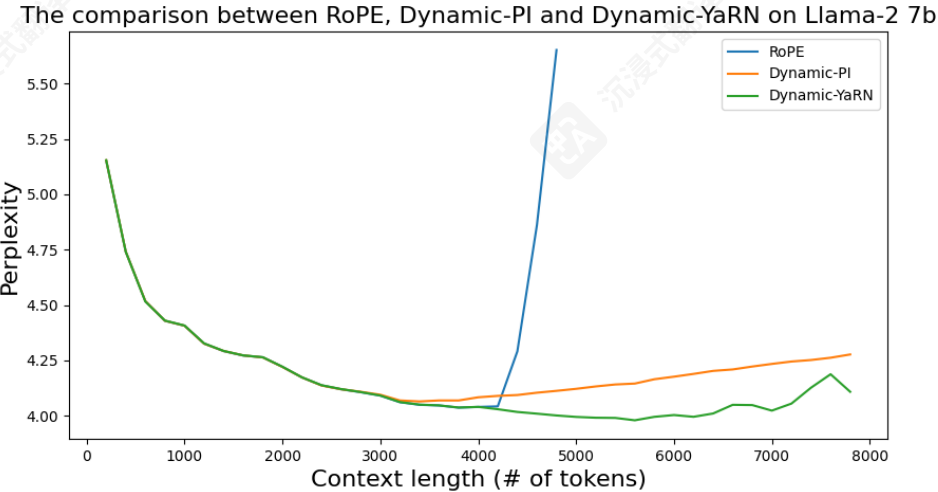


图5: RoPE、Dynamic-PI和Dynamic-YaRN在Llama 2上对长GovReport样本的比较。该模型尚未针对长上下文进行微调。

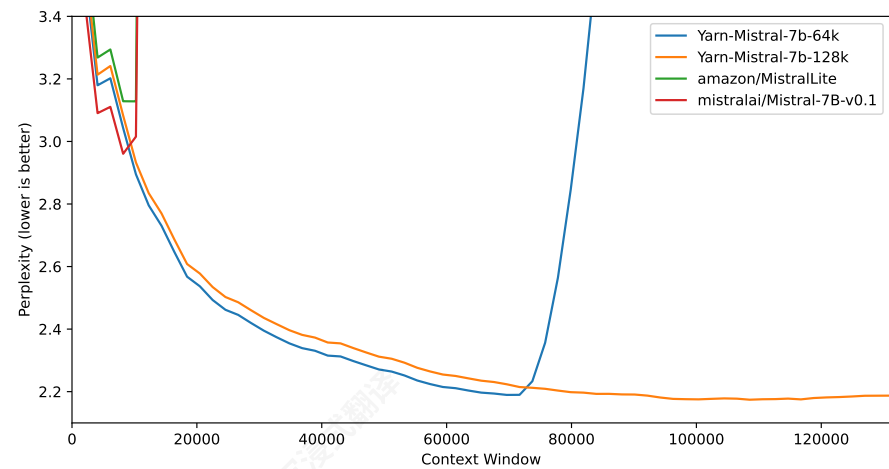


Figure 6: Sliding window perplexity ($S = 256$) of ten 128k Proof-pile documents truncated to evaluation context window size

B.4 Mistral

We additionally extended the Mistral 7B v0.1 model [20], which broadly follows the Llama architecture. For Mistral we trained a 64k context window model ($s = 8$) for 1000 steps using 16k sequence lengths with a constant learning rate of 1×10^{-6} . The model’s sliding window attention size was set to the context window size, effectively disabling sliding window attention. We then trained for an additional 500 steps at $s = 16$ to arrive at a 128k context window model. The training data was a mix of the pre-train and fine-tune splits of Together Computer’s Long-Data Collections [3].

We evaluated the models following the same procedure as described in 4.3.1, comparing against the base v0.1 model and MistralLite [1], an NTK-aware ($\theta = 1M$) version of v0.1. The results (Figure 6 and Table 6) were consistent with those of the Llama family of models.

Model Size	Model Name	Context Window	Extension Method	Evaluation Context Window Size				
				4096	8192	16384	65536	131072
7B	Mistral v0.1	8k	-	3.09	2.96	36.8	$> 10^3$	$> 10^3$
7B	MistralLite	16k	NTK	3.26	3.13	47.3	$> 10^3$	$> 10^3$
7B	YaRN ($s = 8$)	64k	YaRN	3.18	3.04	2.65	2.20	57.4
7B	YaRN ($s = 16$)	128k	YaRN	3.21	3.08	2.68	2.24	2.19

Table 6: Sliding window perplexity ($S = 256$) of ten 128k Proof-pile documents truncated to evaluation context window size

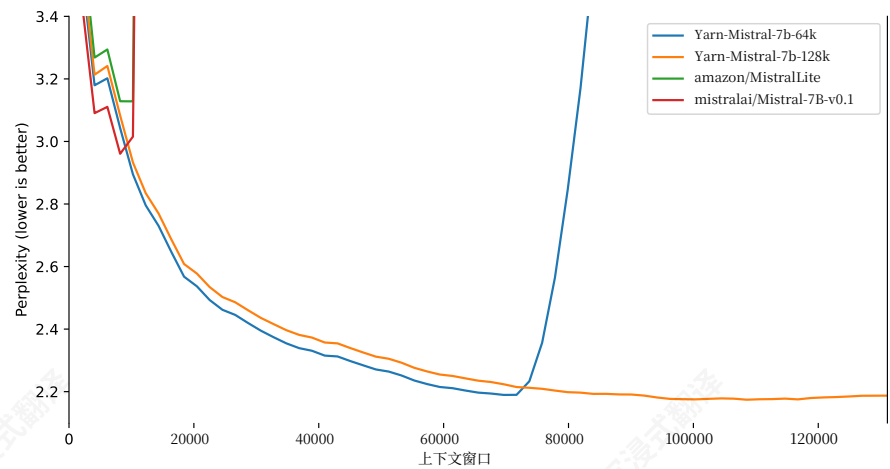


图6: 十个128k证明堆文档的滑动窗口困惑度 ($S = 256$), 截断至评估上下文窗口大小

B.4 Mistral

我们此外扩展了Mistral 7B v0.1模型 [20], 该模型广泛遵循Llama架构。对于Mistral, 我们使用16k序列长度和恒定学习率 1×10^{-6} , 在1000步内训练了一个64k上下文窗口模型 ($s = 8$)。该模型的滑动窗口注意力大小设置为上下文窗口大小, 有效地禁用了滑动窗口注意力。然后我们在 $s = 16$ 处再训练了500步, 以得到一个128k上下文窗口模型。训练数据是Together Computer的Long-Data集合 [3]的预训练和微调拆分混合。

我们按照4.3.1中描述的相同步骤评估了模型, 并与基础v0.1模型和MistralLite [1], 的NTK感知版本v0.1的结果(Figure 6和表6)与Llama模型家族的结果一致。

模型 Size	模型 Name	上下文 窗口	扩展 方法	评估上下文窗口大小				
				4096	8192	16384	65536	131072
7B	Mistral v0.1	8k	-	3.09	2.96	36.8	$> 10^3$	$> 10^3$
7B	MistralLite	16k	NTK	3.26	3.13	47.3	$> 10^3$	$> 10^3$
7B	YaRN ($s = 8$)	64k	YaRN	3.18	3.04	2.65	2.20	57.4
7B	YaRN ($s = 16$)	128k	YaRN	3.21	3.08	2.68	2.24	2.19

表6: 十个128k证明堆文档的滑动窗口困惑度($S = 256$), 截断至评估上下文窗口大小